ASSIGNMENT - 1

① Solve by recurrence at $T(1) = 1$

(i) $T(n) = 4T(n/3) + n$

(ii) $T(n) = 9T(n/3) + n^2$

(i) $T(n) = 4T(n/3) + n$

$= 4(4T(n/9) + n/3) + n$

$= 16T(n/9) + 4n/3 + n.$

$$\Rightarrow T(n) = 4^k T\left(\frac{n}{3^k}\right) + n\sum_{i=0}^{k-1}\left(\frac{4}{3}\right)^i$$

let $\frac{n}{3^k} = 1 \quad = \quad n = 3^k$

$\Rightarrow \log_3 n = k$

$\because T(1) = 1$

$$T(n) = 4^{\log_3^n} + n\left(\frac{(4/3)^{\log_3^n} - 1}{(4/3) - 1}\right)$$

$\Rightarrow 4^{\log_3^n} = n^{\log_3^4}$

$$T(n) = n^{\log_3^4} + 3n\left[\left(\frac{4}{3}\right)^{\log_3^n} - 1\right]$$

$\Rightarrow T(n) = \Theta(n^{\log_3^4})$

(ii) $T(n) = 9T(n/3) + n^2$, $T(1) = 1$

$$T(n) = 9\left[9T\left(\frac{n}{9}\right) + \left(\frac{n}{3}\right)^2\right] + n^2$$

$$T(n) = 81T(n/9) + n^2 + n^2$$

$$T(n) = 81T(n/9) + 2n^2$$

$$\Rightarrow T(n) = 9^{k}T\left(\frac{n}{3^k}\right) + 2kn^2$$

let $\frac{n}{3^k} = 1$, $k = \log_3 n$

$\because T(1) = 1$

$$T(n) = 9^{\log_3 n} + n^2 \log_3 n$$

$\because 9^{\log_3 n} = n^2$

$$\rightarrow T(n) = n^2 + n^2 \log_3 n$$

$$\therefore T(n) = \Theta(n^2 \log n)$$

(2) List any four applications of stacks and queues.

A) Applications of stacks :-

(1) used in recursion known as recursive stack.

(2) used for infix to postfix conversion

(3) used in parenthesis matching

Applications of queues :-

- works based on the principle of first in First Out which is known as FIFO.

(1) used in cpu scheduling where it manages the processes waiting for CPU execution.

(2) used in breadth first search

(3) stores the data packets which are to be transmitted.

(3) Write a function to implement union operations and Intersection on 2 instances of SLL - resultant list should contain unique elements.

A) List* list_union(List *list1, List *list2, List *list3){

```
Node *t1 = list1 -> head;
Node *t2 = list2 -> head;
while(t1 != NULL){
    list_add(list3, t1 -> data);
    t1 = t1 -> next;
}
Node *t3 = list3 -> head;
while(t2 != NULL){
    int flag = 0;
    while(t3 != NULL){
        if(t2 -> data == t3 -> data){
            flag = 1;
            break;
        }
        else{
            t3 = t3 -> next;
        }
    }
    if(!flag){
        list_add(list3, t2 -> data);
    }
    t2 = t2 -> next;
}
return list3;
}
```

```
List * list_intersect(List *l1, List *l2, List *l4){
  Node *t1 = l1->head;
  while(t1 != NULL){
    int flag = 0;
    Node *t2 = l2->head;
    while(t2 != NULL){
      if(t1->data == t2->data){
        flag = 1;
        t2 = t2->next;
        break;
      }
    if(flag){
      Node *t4 = l4->head;
      int f = 0;
      while(t4 != NULL){
        if(t4->data == t1->data){
          t4 = t4->next;
          f = 1;
          break;
        }
        t4 = t4->next;
      }
      if(!f){
        list_add(l4, t1->data);
        f = 0;
      }
      flag = 0;
    }
    t1 = t1->next;
  }
  return l4;
```

```
List * list_add (List *list, int32_t data){
    Node *node = list_node_new (data);
    if (list->head == NULL){
    list->head = node;
    list->tail = node;
     ++ list->length;
}
else{
    list->tail->next= node;
    list->tail= node;
    ++ list->length;
}
return list;
}
```

④ Design data structure to store student data in a single linked list. Write a function to insert and delete an element in the list. Write a function to deallocate all the memory.

A) student.h

```
#ifndef _INCLUDED_SLIST_
#define _INCLUDED_SLIST_
#include <stdio.h>
#include <stdint.h>

typedef struct _node_ Node;
typedef struct _list_ List;

struct _node_ {
    long long reno;
    char name [50];
    float marks 1;
    node *next;
};
```

```
struct _list_{
    Node *head;
    Node *tail;
    uint32_t length;
};

List* list_new();
List* list_student_add(List *list);
List* list_student_remove(List *list);
List* list_free(List *list);

#endif
```

Student.c

```
#include "student.h"
#include <stdlib.h>
#include <stddef.h>
#include <abbrev.h>
#include <limits.h>

List* list_new(){
    List *list;
    list = (List*) malloc(sizeof(List));
    list->head = NULL;
    list->tail = NULL;
    list->length = 0;
    return list;
}
```

```
Static Node* list_new_node(){
    Node *node = (Node*) malloc (sizeof (Node));
    printf ("Enter the name");
    scanf ("%48[^\n]", node->name);
    printf ("Enter RNO :");
    scanf ("%lld", &node->rno);
    printf ("Enter marks");
    scanf ("%f", &node->marks1);
    node->next = NULL;
    return node;
}

List* StudentAdd(List *list){
    Node *node = list_new_node();
    if (list->head == NULL){
        list->head = node;
        node->next = NULL;
    }
    else{
        node->next = list->head;
        list->head = node;
    }
    ++list->length;
    return list;
}
```

```
List * Steedent_Remove (List *list){
  if (list->head == NULL)
    printf ("In No Data");
  else{
  printf ("Enter the rono :");
  long long Key; bcanf ("%lld", &Key);
  Node *temp = list->head;
  Node *cur = NULL;
  if(Key = temp->rono){
   list->head = temp->next;
   free (temp);
   printf ("REMOVED (n");
   list->length --;
   return list;
  }
  while (temp != NULL){
      if (temp->rono == Key){
          cur->next = temp->next;
          temp->next = NULL;
          free (temp);
          printf ("REMOVED \n");
          -- list->length;
          break;
      }
  else{
    cur = temp;
    temp = temp->next;
  }
```

```
if (temp == NULL){
    printf ("\n NOT FOUND\n");
    break;
}
}

return list;

list = list_free (list);
node = (int *)p; if (about (list->head == list->tail));
list = list->head;
list->head = NULL;
list->tail = NULL;
while (list != NULL){
    p = list;
    list = list->next;
    free (p);
    list->length;
}
free (list);
}
return list;
}
```

(6) Reverse a given LinkedList, display Nth node from the end.

```
A) Libt* blibt_revr(Libt *tebl){
    if(libl->head==NULL)
        libl->tail=NULL;

elbl{
    Node *ceekk=lbst->head;
    Node *plev=NULL,*next=NULL;
    lbst->tail=libl->head;
    cehile(ceekk!=NULL){
        next=ceekk->next;
        ceekk->next=plev;
        plev=ceekk;
        ceekk=next;
    }
    libt->head=plev;
    }
    reteern libt;

}
void blibt_nth_node(Libt *libt){
    if(libl->head==NULL){
        lebl->tail=NULL;
}
elbl{
    ert i=1,n;
    plenitf("Enter Nth node\n");
    bcanf("%d",&n);
```

```
if (n < 1 || n > list->length){
   printf("INVALID SIZE");
}
else{
  Node *temp = list->head;
  while(i < list->length - n + 1){
     temp = temp->next;
     i++;
  }
  printf("%th node : %d", temp->data);
}
}
```